COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 20, 2026	**DATE SUBMITTED:**	Oct 20, 2026
NAME:	Corro, Yazinel Obias, Jave Joaquin	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 14.1 ALGORITHM COMPLEXITY

1. PROCEDURE OUTPUT

	Best Case	Worst Case	Analysis
Algorithm 1 (Selection Sort)	$O(n^2)$	$O(n^2)$	routine_smallest always goes through the rest of the array, so it does about n(n-1)/2 comparisons regardless of the input order. At most n-1 swaps. Space O(1).
Algorithm 2 (Quick Sort + Insertion Sort)	O(n)	$O(n^2)$	The code partitions once using the last element as pivot and then insertion sorts the left and right sublists. In the best case, where the data is already sorted, insertion sort needs only about n comparisons, so the best case is O(n). worst case: data is reversed, insertion sort has to shift almost all elements worst case is O(n^2). It does not recurse, so it uses O(1) extra space.

Table 14-1. Best and Worst-Case Analysis using Theoretical Tools

	Algorithm 1	Algorithm 2
Best Case	n=100: 13.06 µs n=1,000: 1,203.8 µs n=10,000: 120,777 µs	n=100: 1.35 µs n=1,000: 12.84 µs n=10,000: 173.0 µs
Worst Case	n=100: 14.22 µs n=1,000: 1,286.0 µs n=10,000: 130,936 µs	n=100: 11.34 µs n=1,000: 1,107.3 µs n=10,000: 113,566 µs

Table 14-2. Best and Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Contrast and compare the two searching techniques used in class. The required deliverable for this activity would be theoretical and empirical analysis.

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	0.0002 ms	0.0048 ms
10,000	0.0003 ms	0.0465 ms
100,000	0.0005 ms	0.4820 ms

Pseudocode Analysis

```
FUNCTION BinarySearch(arr, n, target):
     low = 0
     high = n - 1

     WHILE low <= high DO
          mid = low + (high - low) / 2

          IF arr[mid] == target THEN
               RETURN mid
          ELSE IF arr[mid] < target THEN
               low = mid + 1
          ELSE high = mid - 1
          END IF
     END WHILE

     RETURN -1
END FUNCTION
```

Time and Space Complexities

- **Time Complexity:**
  - **Best Case:** O(1) - The target item is located at the very first index.
  - **Worst Case:** O(n) - The target item is at the last position or not present, requiring n checks.
- **Space Complexity:** O(1) - Requires no extra memory allocation beyond simple counter variables.

Analysis	
Corro	Based on our test results, Binary Search is definitely way faster than Linear Search when the array gets bigger. For $N = 1,000$, Linear Search took $0.0048ms$ while Binary Search was super quick at $0.0002ms$. The difference really shows at $N = 100,000$, where Linear Search jumped all the way to $0.4820ms$ because it has to check elements one by one, which follows $O(n)$ time complexity. On the other hand, Binary Search only took $0.0005ms$ because it keeps cutting the search area in half each step, matching the $O(\log n)$ worst-case theoretical complexity. Overall, Binary Search is much more efficient for large datasets, as long as the array is already sorted beforehand.
Obias	The data we gathered matches what we learned in theory. Linear Search has a worst-case time of $O(n)$ since it might have to scan the entire array if the target is at the end or missing. Binary Search improves this to $O(\log n)$ by dividing the array range in half every iteration. Looking at the execution times, both algorithms took very little time for $N = 1,000$, but Linear Search slowed down significantly as N scaled up to $100,000$. In terms of space, both use $O(1)$ extra memory because they just use simple variables to track index positions. Therefore, Binary Search is the far better choice when dealing with large inputs, provided we can keep the data sorted.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.

I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.